[UORA · DEMO VIDEO]

Shooting Script

Scene-by-scene guide · ~3 minutes 30 seconds

This script is built to be recorded in a single uninterrupted take with the live UORA stack running. Every command, every URL, every click is rehearsed and timed. The voice-over is what you read; the on-screen action is what you do.

FORMAT	1920 × 1080 · 60 fps · screen recording
AUDIO	Voice-over recorded separately, 48 kHz mono
RUNTIME	3 min 30 s (10 scenes)
DELIVERABLE	MP4 · H.264 · ≤ 500 MB
BACKDROP	Dark macOS theme · hide notifications · DND on
STACK NEEDED	All 6 UORA services up at recording time
DRY RUN	Submit dummy_engine.py once before take so MinIO is warm

—— Read the voice-over aloud as if you're explaining UORA to a CTO who has never seen it. ——

Pre-Production Checklist

Before pressing record, verify every item below. If any check fails, fix it first — re-shooting is more expensive than waiting.

#	Check	Command / location
1	Backend running on :8000	`curl -sf localhost:8000/health`
2	Frontend running on :3000	`open http://localhost:3000`
3	Reference engine running on :8081	`curl -sf localhost:8081/health`
4	Local builder consuming build_queue	`ps aux | grep local_builder`
5	Benchmark worker consuming benchmark_queue	`ps aux | grep benchmark.worker`
6	Postgres + Redis + MinIO up	`pg_isready; redis-cli ping`
7	Test account created and password handy	`qa-final@uora.io / QaFinal-2026!`
8	examples/dummy_engine.py is committed	`git status`
9	Browser DND on, notifications muted	System Settings
10	Screen recorder set to 1920×1080 / 60 fps	QuickTime or OBS

Scene Breakdown

10 scenes, ~3:30 total. Time stamps are cumulative; the amber figures are the planned end of each scene.

SCENE 01 — **Cold open — the hook** — **0:15**

LOCATION: UORA landing page · http://localhost:3000

ON SCREEN

`Slow-pan over the landing hero. Animated latency stream on the right is oscillating. Cursor still.`

VOICE-OVER

> *"Every electronic exchange runs on a matching engine. A millisecond too slow, a single out-of-order fill, and someone loses real money. So how do you actually prove your engine is fast — and correct — before it sees live traffic? This is UORA."*

Director note *— No talking head. Pure screen recording. Let the animation breathe.*

SCENE 02 — Tour the landing page — 0:35

LOCATION: Same page, scrolling down

ON SCREEN

Smooth-scroll past the hero stats, past the six-stage pipeline, into the feature cards: Hardened Sandbox, Distributed Bot Fleet, Nanosecond Telemetry, Live Leaderboard.

VOICE-OVER

> *"UORA is a benchmarking platform built for high-frequency matching engines. Upload your code; it's sandboxed, hit with a distributed bot fleet replaying real LOBSTER market data, and ranked on a live leaderboard. Every result is reproducible."*

Director note — *Scroll at a constant 200 px/s so the camera doesn't jerk.*

SCENE 03 — Architecture deep-dive — 0:55

LOCATION: Scroll to 'Four decoupled layers' section

ON SCREEN

Pause on the colored layer pills. Mouse-hover the individual component chips so they get the cyan highlight.

VOICE-OVER

> *"The platform is split into four horizontal layers. Submission and sandbox at the top, then benchmark and validation, telemetry and scoring, and the live UI. They talk only by Redis Streams and Pub/Sub — no shared memory, no hidden coupling."*

Director note — *Move the cursor deliberately — one chip per beat.*

SCENE 04 — Sign in with a real account — 1:15

LOCATION: http://localhost:3000/auth

ON SCREEN

Click 'Launch Console'. Auth page loads. Type qa-final@uora.io and the password. Click 'Sign In'.

VOICE-OVER

> *"The console is gated. Real submissions need a real account. Email and password — or Google OAuth."*

Director note — *Use the typing-clip plugin so the password doesn't show. Pre-warm Chromium so the page loads instantly.*

SCENE 05 — The dashboard, empty — 1:30

LOCATION: http://localhost:3000/dashboard

ON SCREEN

Land on dashboard. Hover the four KPI cards (Active Submissions, Top Score, Best P99, Anomalies). Pause on each.

VOICE-OVER

> *"This is the operator console. Empty for now — there are no scored engines on this team. Top score, best p99, anomalies — all dashes. The moment we submit something, it changes."*

Director note — *Make sure you signed out and cleared state before the take so the dashboard is genuinely empty.*

SCENE 06 — Upload a real engine — 2:00

LOCATION: Submit panel

ON SCREEN

Drag examples/dummy_engine.py into the drop zone. 'Python · 3.13' badge appears. Click 'Submit Engine'. Watch Recent Submissions: QUEUED → BUILD → DEPLOY → BENCHMARK pipeline fills with green check marks.

VOICE-OVER

> *"Drag in a Python matching engine. Two hundred lines of stdlib — price-time priority, self-trade prevention, the full HTTP contract. The platform detects the language, queues the build, and shows the pipeline tracker."*

Director note — *The dropped file MUST be the latest dummy_engine.py — the DELETE-cancel fix is what gets it to score. Pre-stage it on the desktop.*

SCENE 07 — Live build log — 2:20

LOCATION: Right side of the dashboard

ON SCREEN

Pan to the 'Live Build Log' panel. Real lines appear: `$ python3 --version`, `→ Entry: dummy_engine.py`. Color-coded — green commands, gray info.

VOICE-OVER

> *"The right panel is the real compiler output, not a scripted demo. Python 3.13 boots, the engine binds to a free port, the bot fleet starts firing."*

Director note — *If the build log is empty, sign out, sign back in, retry. The log only renders on a fresh submission.*

SCENE 08 — Score reveal — 2:45

LOCATION: Submit panel after ~15 s

ON SCREEN

`Recent Submissions row flips to SCORED. The 4-metric grid appears: Score (cyan), P99 Latency, Throughput, Correctness (green).`

VOICE-OVER

> *"And that's a real score: composite of two hundred and twenty-eight, p99 around a hundred and twenty milliseconds, throughput at twenty-eight thousand orders a second, correctness one hundred percent. Every number is from real HTTP traffic against a real running process."*

Director note — *The exact numbers will vary ±5% each run. Don't re-take if they differ — the point is they're real.*

SCENE 09 — Leaderboard + downloadable PDF report — 3:10

LOCATION: Sidebar → Leaderboard, then Reports

ON SCREEN

`Click 'Leaderboard'. Single row at rank 1 — QAFinal team. Click 'Reports'. Click 'Download PDF Report'. Browser shows the file save dialog briefly.`

VOICE-OVER

> *"The leaderboard updates the moment the score lands. Click into Reports and you get the full performance audit — tail latency, fill correctness, anomaly status, and a downloadable PDF that ships with the source."*

Director note — *The PDF really downloads — don't fake it. Show the macOS download bezel.*

SCENE 10 — Close — 3:30

LOCATION: Back to landing page

ON SCREEN

`Cmd-click 'UORA' logo to open landing in a new tab. Type-in overlay: github.com/vansh7266/uora-platform. Fade to UORA mark on black.`

VOICE-OVER

> *"UORA. Compile, sandbox, replay, validate, score — all in under thirty seconds, with every number reproducible. Source on the screen. Thanks for watching."*

Director note — *Render the closing card in post — don't try to record the fade live.*

Post-Production Notes

Music. Subtle, ambient, no sub-bass that masks the voice. Recommended: a 90 BPM lo-fi loop at -22 LUFS.

Cuts. Hard cut between scenes — no fades. Total cuts ≤ 12.

On-screen text. Two captions only: 'UORA' at 0:01 and 'github.com/vansh7266/uora-platform' at 3:25. Use the same Helvetica Bold as the dashboard.

Cursor. Use Mouseposé (macOS) to halo the pointer during clicks so the viewer's eye follows it.

Audio leveling. Normalize voice to -16 LUFS. Compress 3:1, gentle ratio. Truepeak ceiling -1 dB.

Export. 1080p60, H.264 high profile, CRF 18, audio AAC 192 kbps. Target size: 80-120 MB.

—— End of script · go shoot ——